

Assignment: Recurrent Neural Networks

Topic: Sequential Modeling, Backpropagation Through Time (BPTT), and Gradient Dynamics.

By

Dansi Acharya.

Problem Context.

Consider a RNN processing a sequence of three time-step inputs (x_1, x_2, x_3) . The hidden state at each time step t is updated via the recurrence relation

$$h_t = f(x_t, h_{t-1})$$

The scalar loss L is evaluated at the final step as a function of a terminal hidden state h_3 :

$$L = L(h_3)$$

Questions:

(1) Forward Pass:

Explain how the RNN processes the sequence (x_1, x_2, x_3) . Draw an unrolled computational graph illustrating the dependency flow among the inputs x_t , hidden states h_t , and the final loss L .

Ans. $\rightarrow$

The forward pass begins with an initial hidden state h_0 , which is commonly initialized to a zero vector. The RNN processes the inputs sequentially.

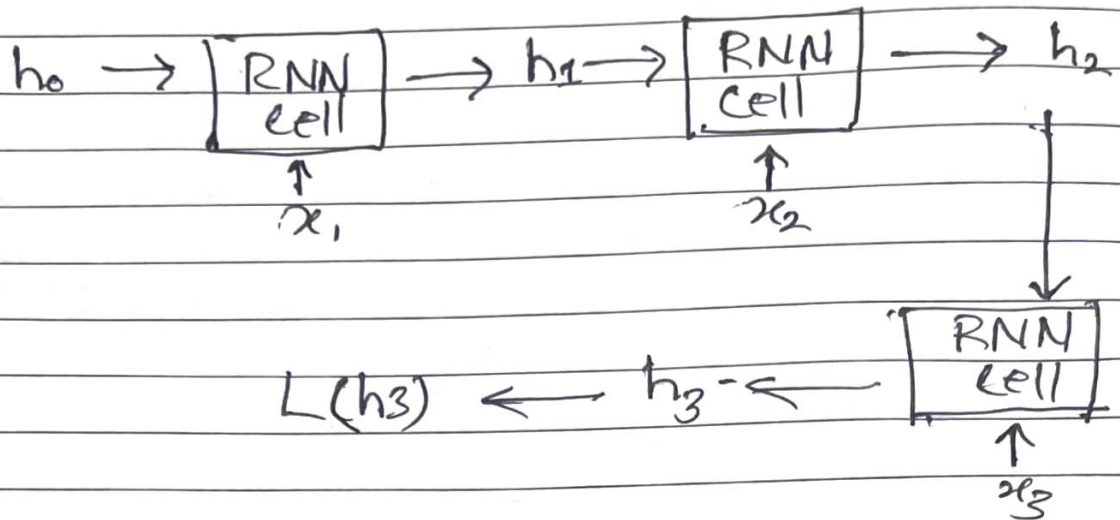
$$\begin{aligned} h_1 &= f(x_1, h_0) \\ h_2 &= f(x_2, h_1) \\ h_3 &= f(x_3, h_2) \end{aligned}$$

Date _____
Page _____

Finally, the scalar loss is computed from the terminal hidden state.

$$L = L(h_3)$$

The unrolled computational graph is



Equivalently, the dependency path can be written as:

$$x_1 \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow L$$

$\uparrow \quad \quad \uparrow \quad \quad \uparrow$
 $h_0 \quad x_2 \quad x_3$

Although, the unrolled graph displays three RNN cells, these are three uses of the same cell with the same trainable parameters.

② Hidden state Dynamics.

Explain the functional Role of the hidden state h_t . How does h_t accumulate and carry context from preceding time steps across the sequence?

Answer The hidden state h_t acts as the RNN's dynamic temporal memory. It summarizes information from the inputs observed up to the time step t .

Because, $h_t = f(x_t, h_{t-1})$,

the current state depends directly on x_t and h_{t-1} . However, h_{t-1} already depends on earlier inputs. For example,

$$h_3 = f(x_3, h_2) = f(x_3, f(x_2, f(x_1, h_0)))$$

Therefore, h_3 can indirectly contain information from x_1 , x_2 and x_3 .

The hidden state does not normally store every previous input exactly. Instead, during training, the RNN learns which ~~historical~~ historical features should be retained, transformed, or forgotten. Its fixed dimensionality provides a compact representation of the relevant sequence context.

③ Backpropagation Through Time

Ans → Since, the loss L depends on h_3 , the backward pass begins by computing:

$$\frac{\partial L}{\partial h_3}$$

Because h_3 depends on h_2 , the gradient for h_2 is:

$$\frac{\partial L}{\partial h_2} = \frac{\partial L}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2}$$

Similarly, because h_2 depends on h_1 ,

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1}$$

The complete backward dependency is:

$$L \rightarrow h_3 \rightarrow h_2 \rightarrow h_1$$

This process is called backpropagation through time because RNN is unrolled across its temporal steps and gradients are propagated backward through those steps. Time itself is not a trainable variable, the name refers to moving backward through the time-indexed computational graph.

② Chain Rule Derivation

Using the multivariable chain rule, derive the explicit analytical expression for the gradient.

$$\partial L / \partial h_1$$

Show each step of the expansion through intermediate hidden states h_2 and h_3 .

Answer $\Rightarrow$ The loss has the dependency chain:

$$h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow L$$

First, the gradient at h_3 is:

$$\frac{\partial L}{\partial h_3}$$

Applying the multivariable chain rule to h_2

$$\frac{\partial L}{\partial h_2} = \frac{\partial L}{\partial h_3} \times \frac{\partial h_3}{\partial h_2}$$

Again, applying it to h_1 :

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1}$$

Substituting the expression for $\partial L / \partial h_2$ gives:

$$\left[\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \right]$$

For vector-valued hidden states, these hidden-to-hidden derivatives are Jacobian matrices. Using column-gradient notation, the expression is commonly written as:

$$\nabla_{h_1} L = \left(\frac{\partial h_2}{\partial h_1} \right)^T \cdot \left(\frac{\partial h_3}{\partial h_2} \right)^T \nabla_{h_3} L$$

For the vanilla RNN.

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

The local Jacobian is:

$$\frac{\partial h_t}{\partial h_{t-1}} = D_t W_{hh}$$

$$\text{where } D_t = \text{diag}(1 - h_t^2)$$

$$\nabla_{h_1} L = W_{hh}^T D_2 W_{hh}^T D_3 \nabla_{h_3} L$$

This product shows how the gradient is repeatedly transformed while moving backward through the recurrent states.

⑤ Intermediate Activations & Memory.

During the forward pass, hidden states (h_1, h_2, h_3) are computed sequentially. Explain why and how these intermediate activation vectors must be stored in memory for use during the backward pass.

Answer During the forward pass, the RNN computes intermediate states such as h_1, h_2, h_3 . These states are required during the backward pass because local derivatives depend on the values produced during the forward computation.

For example, if
 $h_t = \tanh(ax)$,
 then,

$$\frac{\partial h_t}{\partial ax} = 1 - h_t^2.$$

Thus, calculating the backward gradient requires the previously computed value of h_t , or equivalent saved information.

Automatic Differential systems such as PyTorch construct a computational graph during the forward pass and save the tensors for derivative calculations. During backward, the graph is traversed in reverse order. Consequently, the memory required for full BPTT generally grows with sequence length.

⑥ Parameter Sharing

Explain why the same weight parameters are shared across the every time step in an RNN. How does parameter sharing affect gradient computation and parameter updates during BPTT?

Answer An RNN uses the same weights and biases at every time step. For a vanilla RNN, the shared recurrence is

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

Parameter sharing has several advantages.

- It allows the same transformation to be applied at every sequence position.
- It prevents the number of parameters from increasing with sequence length.
- It allows the model to process sequence of different lengths.
- It helps the model detect temporal patterns regardless of their exact position.

Because the same parameter affects the loss through every time step, its gradient is the sum of its contributions across time. For a shared parameter θ

$$\frac{\partial L}{\partial \theta} = \sum_{t=1}^T \frac{\partial L}{\partial \theta} \quad \text{contribution from time } t$$

⑦ Vanishing and Exploding Gradients

Explain why gradients can exponentially diminish (vanish) or grow (explode) when BPTT is performed over long temporal sequence.

Answer During BPTT, the gradient for an early hidden state contains a product of many hidden-to-hidden Jacobian matrices.

$$\nabla_{h_k} L = \left(\prod_{t=k+1}^T J_t^T \right) \nabla_{h_T} L,$$

Where, $J_t = \frac{\partial h_t}{\partial h_{t-1}}$

The magnitude of this product can be approximately bounded by

$$\|\nabla_{h_k} L\| \leq \left(\prod_{t=k+1}^T \|J_t\| \right) \|\nabla_{h_T} L\|$$

If the relevant Jacobian norms or singular values are repeatedly smaller than 1, the product decrease approximately exponentially. The gradients reaching early time step become extremely small, producing the vanishing gradient problem. The model then struggle to learn long-range dependency.

For a tanh RNN,

$$J_t = D_t W_{hh}$$

and the tanh derivatives satisfies:

$$0 < 1 - h_t^2 \leq 1$$

When tanh is saturated near -1 or 1 , its derivatives is close to zero, which increases the likelihood of vanishing gradients.

Common remedies includes

- LSTM or GRU architecture.
- Gradient clipping for exploding gradients.
- Careful parameter initialization.
- Normalization methods
- truncated BPTT
- Shorter dependency paths and gated memory mechanism.

⑧ Practical implementation (PyTorch)

- Using PyTorch, implement a simple custom RNN cell or module that processes a sequence of at least 5 time steps (T=5). Compute a scalar loss, call `.backward()` and print the gradient tensor (`.grad`) for each model parameter. Explain what these parameter gradients physically represent.

Answer The program is implemented within the folder- Assignment-3 of NLP assignment repo in github. This program implements a vanilla RNN that processes a sequence of five time steps. At every step, the hidden state is calculated using the current input and previous hidden state.

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

The final hidden state is transformed into an output, and mean squared error is used to produce a scalar loss. Calling `loss.backward()` causes PyTorch to apply BPTT automatically.

The `.grad` tensor associated with each parameter contains,

$$\frac{\partial L}{\partial \theta}$$

Date _____
Page _____

where θ is the parameter. Each element measures how a small increase in the corresponding parameter element would locally change the loss.

A positive gradient means increasing that parameter would locally increase the loss, whereas a negative gradient means increasing it would locally decrease the loss. The gradient's magnitude indicates local sensitivity. Because recurrent parameters are shared, gradients such as w_{-hh} .weight.grad accumulate contributions from every time step at which the parameter was used.

An optimizer uses these gradients to update the parameters. For standard gradient descent.

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \frac{\partial L}{\partial \theta}$$